

DigiRoutes

Share your exact doorstep.

What it is	An Android app, website and backend that turns any place in India into a short DIGIPIN code, and lets people save, share and navigate to exact doorsteps with address cards, photos and private QR links.
App version	1.6.0 (Android, built with Flutter)
Website / API	https://digiroutes.vercel.app (Next.js on Vercel)
Source code	GitHub: digiroutes_app (mobile app) and digiroute (website + backend)
Stack	Flutter · Riverpod · Next.js · MongoDB Atlas · Cloudinary · OpenRouteService · GitHub Actions · Vercel
Size	~47 app source files (~11,000 lines) · ~12,000 lines of web/backend code · 28 API endpoints · 39 automated tests

How to use: Parts 1–3 explain the idea and design (everyone reads these). Part 4 is the feature catalogue — one self-contained section per feature. Part 10 suggests how to split the features between presenters, and Part 11 is a 12-minute demo script.

Contents

Part 1 — The problem and our solution

Part 2 — Architecture (how the pieces fit)

Part 3 — The core idea: how DIGIPIN works

Part 4 — Feature catalogue

- F1 — Splash and onboarding
- F2 — Sign up and log in
- F3 — Themes, sound and haptics
- F4 — DIGIPIN engine (encode / decode)
- F5 — Home: map and "Get My DIGIPIN"
- F6 — Compass tab (DIGIPIN tools)
- F7 — "Where to?" route planner
- F8 — Create a card
- F9 — My Cards: list, search, filters, favourites, pagination
- F10 — Edit, delete and automatic photo cleanup
- F11 — Card detail
- F12 — Private share links (a key feature to explain well)
- F13 — QR codes
- F14 — Scan tab (camera, gallery, typed code)
- F15 — Open links in the app, web fallback, rich share
- F16 — Launcher shortcuts

F17 — Profile

F18 — Settings and side menu

F19 — Delete account

F20 — In-app updates

F21 — Secure session storage

F22 — Network errors and offline behaviour

F23 — Server-side protections (summary)

F24 — The public website

F25 — Admin panel (website only)

F26 — Creating the super admin safely

F27 — Testing

F28 — CI/CD, releases and versioning

F29 — Project structure (where things live)

Part 5 — Backend API at a glance

Part 6 — Database design (MongoDB)

Part 7 — Problems we hit and how we solved them

Part 8 — Limitations and future work

Part 9 — Glossary

Part 10 — Suggested division of features

Part 11 — A 12-minute live demo script

Part 12 — Questions the teacher may ask (general)

Part 1 — The problem and our solution

1.1 The problem

In India, a normal postal address ("near the blue gate, behind the temple, 2nd lane") is often vague. Delivery people, guests and cab drivers lose time finding the right *door*. A map pin helps, but it is not remembered, not described, and not easy to share.

1.2 DIGIPIN — the idea we build on

DIGIPIN is a 10-character code published by India Post. Every $\sim 4\text{ m} \times 4\text{ m}$ square of India has its own code (example: the point 28.6139° N , 77.2090° E in New Delhi is **39J438TJC7**). It is short, easy to say or write, and it can be turned back into coordinates **without internet**, because it is just maths.

1.3 Our solution — DigiRoutes

DigiRoutes adds everything a bare DIGIPIN is missing:

1. **Address cards** — a DIGIPIN plus a title, entrance photos, a delivery note ("ring twice"), a phone number and a category.
2. **Private sharing** — each card gets its own secret link and QR code; the owner can switch it off, reset it, or make it expire.
3. **Scanning** — open a card by scanning its QR (camera, or a QR image from the gallery) or pasting a link.
4. **Routes** — "Where do you want to go?" shows distance, time and the road route between any two points.
5. **An admin panel** on the website for the people running the service.

1.4 Who uses it

User	What they do
Resident / shop owner	Creates a card for their door, adds a photo and note, shares the link or QR.
Visitor / delivery person	Opens the link or scans the QR → sees photo, note, map; taps Navigate or Call.
Admin	Watches analytics, manages users, removes bad content (website only).

Part 2 — Architecture (how the pieces fit)



2.1 The two repositories

Repo	What it contains
<code>digiroutes_app</code>	The Android app (Flutter/Dart): ~47 source files, ~11,000 lines, 39 automated tests.
<code>digiroute</code>	The website and the backend API (Next.js/TypeScript): ~12,000 lines, 28 API endpoints, 5 database collections.

2.2 Technology stack and *why*

Layer	Choice	Why we chose it
Mobile app	Flutter (Dart)	One codebase, fast UI, rich animations, camera/GPS plugins.
State management	Riverpod	Simple, testable way to share data (user, cards, theme) between screens.
Navigation	go_router	Deep links (open <code>https://.../c/<token></code> straight in the app) and tab navigation.
Backend + website	Next.js (React)	Frontend pages and API routes in one project; deploys to Vercel with one push.

Layer	Choice	Why we chose it
Database	MongoDB Atlas (Mongoose)	Flexible documents suit cards with optional fields; free tier.
Images	Cloudinary	Stores photos on a CDN; the app uploads directly so our server never handles image bytes.
Routing / search	OpenRouteService	Free road-routing and place-search API (based on OpenStreetMap).
Maps	Google Maps embed + OpenStreetMap tiles (<code>flutter_map</code>)	No billing needed.
Auth	JWT (<code>jose</code>) + bcrypt	Stateless sessions; passwords never stored in plain text.
CI/CD	GitHub Actions + Vercel	Every merge to <code>main</code> tests, builds, signs and publishes automatically.

All services we use are on **free plans** — the whole project costs ₹0 to run.

2.3 One request, end to end (example: opening a shared card)

1. A friend scans the QR of your card → it contains `https://digiroutes.vercel.app/c/AbC...` (22 random characters).
2. Android sees the link belongs to our app (verified through `assetlinks.json`) and opens DigiRoutes directly. *(No app installed? The link opens the website instead.)*
3. The app calls `GET /api/cards/shared/<token>` — no login needed.
4. The server checks: token exists? sharing switched on? not expired? If not → "This link isn't available".
If yes it returns **only** what a visitor may see (no owner id, no internal photo ids; phone hidden if the owner chose so) and counts one view.
5. The app shows photo, note, map, and Call / WhatsApp / Navigate buttons.

Part 3 — The core idea: how DIGIPIN works

3.1 The grid

India's bounding box is `lat 2.5° – 38.5°`, `lon 63.5° – 99.5°`. The box is cut into a **4 × 4 grid** of 16 cells. Each cell has a letter from this table:

	col 0	col 1	col 2	col 3
row 0	F	C	9	8
row 1	J	3	2	7
row 2	K	4	5	6
row 3	L	M	P	T

Then the chosen cell is cut into another 4 × 4 grid, and so on — **10 times**. Each level adds one character, so the code has 10 characters.

3.2 Worked example — New Delhi (28.6139° N, 77.2090° E)

Level	Current square	Where the point falls	Character
1	36° × 36° (all of India)	row 1, col 1	3
2	9° × 9°	row 0, col 2	9
3	2.25° × 2.25°	row 1, col 0	J
4 ... 10	keeps shrinking by ÷4 each time	...	438TJC7

Result: `39J438TJC7`. Decoding it gives back `28.613901`, `77.208998` — less than one metre away from where we started.

3.3 Why "about 4 metres"

After 10 divisions by 4, a side is $36^\circ \div 4^{10} = 0.0000343^\circ$, which is ≈ 3.8 m north–south and ≈ 3.4 m east–west at Delhi's latitude. That is the size of one DIGIPIN square.

3.4 Properties worth mentioning

- **Works offline** — encode/decode is pure maths (`lib/logic/digipin.dart` , mirrored in `digiroute/lib/digipin.ts`).
- **Only valid inside India's box**; outside it the app shows a clear error.
- **A DIGIPIN is not secret**. Anyone can compute it from a location. This is why we built *private share links* (feature F12) — a card must never be unlocked by its DIGIPIN alone.

Part 4 — Feature catalogue

How to read each feature: **What** it is → **How** it works → **Tech** (libraries and key files) → **Demo** (steps you can show live) → **Teacher may ask**.

A. Getting started

F1 — Splash and onboarding

What: The first things a new user sees: an animated logo screen, then three swipeable introduction slides.

How: The splash waits ~2 seconds and checks if a login token is saved. Logged-in users go straight to Home; new users see onboarding. The three slides are *drawn in code* (no photos): (1) a DIGIPIN grid with a pin dropping into one square, (2) a stack of address cards with a pulsing favourite star, (3) an animated route with a navigating arrow. The images drift at a different speed from the text while swiping (parallax).

Tech: `flutter_animate`, `CustomPainter` for the route drawing; `lib/ui/screens/splash_screen.dart`, `onboarding_screen.dart`.

Demo: Reinstall/clear data → watch splash → swipe the three slides → tap *Get Started*.

Teacher may ask: *Why draw illustrations in code instead of using images?* → Smaller app, perfect on every screen size, works in light and dark theme, and no copyright issues.

F2 — Sign up and log in

What: Create an account with name, email and password; log in later.

How: The app sends the details to `POST /api/auth/signup` or `/login`. The server stores only a **bcrypt hash** of the password (never the password), and returns a **JWT** (a signed token valid for 7 days). The app keeps the token in the phone's secure keystore and sends it with every later request as `Authorization: Bearer <token>`.

Tech: `bcryptjs`, `jose` (JWT), `flutter_secure_storage`; `lib/data/repositories/auth_repository.dart`, `digiroute/app/api/auth/*`.

Protection: login allows 8 wrong attempts per 15 min per account+IP (then HTTP 429); signup 5 per hour per IP.

Demo: Sign up → close the app completely → reopen: you are still logged in.

Teacher may ask: *What is a JWT?* → A token with three parts (header, data, signature). The server signs it with a secret key; it can't be changed without the key, so the server can trust it without looking anything up. *Why not store passwords?* → If the database leaks, hashes can't be turned back into passwords.

F3 — Themes, sound and haptics

What: Light theme by default, plus Dark and Auto (follows the phone). Optional click sound and vibration.

How: The choice is saved on the phone (`shared_preferences`) and applied instantly to the whole app via Riverpod.

Tech: `lib/core/theme/app_theme.dart` , `lib/logic/providers.dart` .

Demo: Settings → Appearance → tap Dark, Light, Auto.

B. Location tools

F4 — DIGIPIN engine (encode / decode)

What: The heart of the app: convert coordinates $\rightleftharpoons$ DIGIPIN.

How: See Part 3. Encoding repeatedly splits the India box into 4×4 cells; decoding walks the same path backwards and returns the *centre* of the final cell.

Tech: `lib/logic/digipin.dart` (app, works offline) and `digiroute/lib/digipin.ts` (server/website). Unit tests check round trips and invalid input.

Demo: In the Compass tab enter `28.6139, 77.2090` → `39J438TJC7` .

Teacher may ask: *Why is the result a little different from my input?* → The code names a 4 m square; decoding returns its centre, so the answer is within about 2–3 m.

F5 — Home: map and "Get My DIGIPIN"

What: A full-screen map with a floating search bar and a button that tells you the DIGIPIN of where you stand.

How: The map is a Google Maps *embed* shown in a web view (no API key). *Get My DIGIPIN* reads the phone's GPS (`geolocator`), encodes it, moves the map there and shows a card with **Copy**, **Share** and **Share Google Maps link**.

Tech: `webview_flutter` , `geolocator` ; `lib/ui/screens/home_screen.dart` , `lib/core/map_widgets.dart` .

Demo: Tap *Get My DIGIPIN* → allow location → copy the code.

Teacher may ask: *What if location permission is denied?* → The app shows a clear message and keeps working; GPS is only needed for this button and for new cards.

F6 — Compass tab (DIGIPIN tools)

What: A utility with three modes: **GPS** (your DIGIPIN now), **Decode** (type a DIGIPIN → see it on the map), **Coords** (type or paste latitude, longitude → get the DIGIPIN).

How: Each mode feeds the same result card showing the DIGIPIN, the coordinates, a *Navigate with Google Maps* button, a **QR code** and **Share**. "Paste Pair" reads `lat` , `lon` from the clipboard.

Tech: `lib/ui/screens/compass_screen.dart` .

Demo: Decode mode → enter a DIGIPIN → map and coordinates appear.

Note for the group: despite its name this tab does **not** show a magnetic compass needle — it is the "find and convert" tool.

F7 — "Where to?" route planner

What: Pick a **start** and a **destination** and see the distance, travel time and road route; works for driving, cycling and walking.

How: Tap the search bar on Home. Each end can be: current location, a saved card, a typed DIGIPIN or link, pasted coordinates, a **place name**, or a scanned QR. The app asks OpenRouteService for the road route (distance, duration and the line to draw) and draws it on an OpenStreetMap map. Separately it always shows the **straight-line distance**, computed offline with the *haversine formula* (great-circle distance on a sphere). *Navigate in Google Maps* hands the trip to Google Maps for turn-by-turn.

Tech: `flutter_map`, `latlong2`, `OpenRouteService` (`api.heigit.org`), `lib/core/routing_service.dart`, `lib/logic/geo.dart`, `route_screen.dart`, `widgets/point_picker_sheet.dart`.

Demo: Home → search bar → destination "India Gate" → watch the route, time and distance → switch Drive/Bike/Walk.

Teacher may ask: *What is the haversine formula?* → It computes the shortest distance between two latitude/longitude points over the Earth's curved surface. *What if the routing service is down?* → The app still shows the straight-line distance and a readable message.

C. Address cards

F8 — Create a card

What: Save a doorstep: location, title, address text, category, delivery note, phone number and up to two entrance photos.

How: The + button in the bottom bar opens the form and grabs your GPS location automatically, so saving is "type a title → Save". Photos (camera or gallery, max **4 MB** each) are uploaded **straight to Cloudinary** using parameters the server signed for you — the server never receives image data. The server validates every field (category from a fixed list, note ≤ 300 characters, phone 7–15 digits).

Tech: `image_picker`, Cloudinary signed uploads (`/api/upload/sign`); `create_card_screen.dart`, `digiroute/app/api/cards/route.ts`, `lib/cardFields.ts`.

Demo: Tap + → wait for the DIGIPIN chip → add title, category, note → add a photo → Save.

Teacher may ask: *Why upload directly to Cloudinary?* → Faster, cheaper (no server bandwidth), and the secret API key never leaves the server.

F9 — My Cards: list, search, filters, favourites, pagination

What: All your cards in a grid with search, filter chips (All, Favorites, each category you use) and a favourite star on every card.

How: Cards load **12 at a time** as you scroll (*cursor pagination*: "give me the next 12 older than this id"). Search and filters run **on the server**, so they cover all your cards, not only the ones already on screen. The favourite star updates instantly (optimistic update) and is rolled back if the server refuses. Counts and category chips come from the server's first-page "facets".

Tech: `lib/logic/providers.dart` (`CardsNotifier`), `dashboard_screen.dart` , `GET /api/cards?q=&category=&favorite=&cursor=` .

Demo: Create several cards → search a word → tap *Favorites* → scroll to load more.

Teacher may ask: *Why not load everything at once?* → Slow and wasteful with hundreds of cards; pagination keeps it fast and light on data.

F10 — Edit, delete and automatic photo cleanup

What: Change any card's details; delete cards; photos never pile up on the server.

How: Location is fixed once created (a different place is a different card). Whenever a photo is replaced or removed, a card is deleted, the profile picture changes, or the account is deleted, the images are **deleted from Cloudinary**. If a save *fails* after photos were uploaded, the app tells the server to discard those uploads (`/api/upload/cleanup`), which only deletes unused images inside the caller's own folder.

Why it matters (security): the server checks every image id belongs to the user's own folder — otherwise someone could claim another user's photo id and get it deleted.

Demo: Delete a card → show that its photo URL no longer opens.

F11 — Card detail

What: The full card: photo carousel, DIGIPIN chip (tap to copy), category, note, map, coordinates and action buttons.

How: Buttons: **Navigate** (opens Google Maps), **Call** and **WhatsApp** (if a phone number is set and not hidden), **QR code**, **Share**. The owner also has a favourite star, an edit/delete menu and the *Share with a link* panel (F12).

Tech: `card_detail_screen.dart` , `url_launcher` (`tel:` , `https://wa.me/...`).

D. Sharing — the privacy design

F12 — Private share links (*a key feature to explain well*)

What: Every card has its own secret web link, e.g. `https://digiroutes.vercel.app/c/Qw3rT...` (22 random characters = 128 bits). Only people who have the link can see the card.

Why we built it: a DIGIPIN can be computed by anyone from a location. In version 1 the public lookup was *by DIGIPIN*, so someone who knew roughly where you live could compute your DIGIPIN and read your photo and phone number. Now the DIGIPIN **never** unlocks card data.

Owner controls (on the card):

- **Share with a link: on/off** — off means the card is completely private.

- **Reset link** — creates a new token; old links and QR codes stop working.
- **Expiry** — Never / 24 hours / 7 days.
- **Hide my phone number** — visitors don't see Call/WhatsApp.
- **"Opened N times"** counter.

How it behaves: an off, expired, reset or unknown link all show the *same* "This link isn't available" page, so nobody can tell which links exist. The shared page is marked *noindex* (not in Google), never cached, and sends no referrer.

Old cards: cards created before this feature keep working through their old DIGIPIN link until the owner taps *Reset link* (grace period, so already-shared links don't suddenly break).

Tech: `digiroute/lib/shareLinks.ts`, `GET /api/cards/shared/[token]`, web page `/c/[token]`, app `card_detail_screen.dart` (`_SharingPanel`).

Demo: Open a card → copy link → open it in a browser (works) → turn sharing off → refresh (blocked) → turn on → *Reset link* → old link dies.

Teacher may ask: *Isn't a link just as guessable?* → No — 128 random bits means 3.4×10^{38} possibilities; guessing is impossible. *What if someone forwards the link?* → We can't stop copying, but the owner can reset or expire it any time.

F13 — QR codes

What: A scannable QR for each card (and for results in the Compass tab).

How: The QR simply contains the card's share link. It is drawn on a white square so it scans reliably in dark mode too; the owner can **share it as a PNG image** or copy the link. Only shown if sharing is switched on.

Tech: `qr_flutter`, `lib/ui/widgets/qr_sheet.dart`.

Demo: Card detail → QR icon → show it to a second phone.

F14 — Scan tab (camera, gallery, typed code)

What: The second tab. Three ways to open a card or location from a code:

1. **Camera** — point at a QR; torch button for dark places.
2. **From gallery** — pick a screenshot or saved QR image (useful when the QR is on the *same* phone).
3. **Type code** — paste a link or type a DIGIPIN.

How: `mobile_scanner` uses Google ML Kit to read the QR. The app then works out what it found: a private card link (`/c/...`), an older card link (`/card/<PIN>`), or a bare DIGIPIN, and opens the right screen. It rejects QR codes that aren't ours. The camera runs **only while the Scan tab is visible**.

Tech: `lib/ui/screens/scan_screen.dart`, `lib/logic/qr_link.dart` (parser, unit tested).

Demo: Scan the QR from F13; then pick a saved QR image from the gallery.

Teacher may ask: *Does it send my camera images anywhere?* → No; recognition happens on the phone.

F15 — Open links in the app, web fallback, rich share

What: Tapping a DigiRoutes link anywhere (WhatsApp, browser...) opens the **app** if installed; otherwise the **website**.

How: Android *App Links*: the app declares it handles `digiroutes.vercel.app/c/...`, and our site publishes `/.well-known/assetlinks.json` with the app's signing fingerprint so Android can verify we own both. On the website, Android browsers show an *Open in app* banner. **Rich share** sends one message with photo, DIGIPIN, address, note, phone, Google Maps link and the private link.

Tech: `AndroidManifest.xml` intent filters, `share_plus`, `lib/core/card_share.dart`, `digiroute/public/.well-known/assetlinks.json`.

F16 — Launcher shortcuts

What: Long-press the app icon → **New card here** and **Scan QR**.

How: Android *app shortcuts* via the `quick_actions` plugin; they navigate straight to the right screen.

Not done: a true home-screen *widget* (needs native Android code) — listed as future work.

E. Account

F17 — Profile

What: A dedicated page: profile picture (camera / gallery / remove, max 4 MB), editable name, read-only email, and counts of your cards and favourites.

How: The picture is uploaded like card photos; when it is replaced or removed the old image is deleted automatically.

Tech: `profile_screen.dart`, `PUT /api/auth/me`.

F18 — Settings and side menu

What: Settings page: account summary, theme tiles with previews, sound & haptics, check for updates, version, delete account, log out. The **side menu** has only account items (Profile, Settings, Share DigiRoutes) with **Log out pinned at the bottom** and always asking for confirmation, so it can't be hit by accident. Navigation items live in the bottom bar and are **not duplicated** in the side menu.

Bottom bar: `Home` · `Scan` · `[+]` · `Cards` · `Compass` — the round `+` in the middle creates a card.

F19 — Delete account

What: Permanently remove your account.

How: Asks for your password. The server then deletes every card, every photo, the profile picture and finally the account itself, and ends all sessions. A wrong password is refused.

Teacher may ask: *Why is this important?* → Privacy rules (such as India's DPDP Act) expect users to be able to erase their data.

F20 — In-app updates

What: The app tells you when a new version exists and installs it.

How: The app is distributed as an APK on GitHub (not the Play Store). On launch it asks the GitHub Releases API for the latest version; if that number is higher than its own, it offers to download the APK (with a progress bar) and hands it to Android's installer. Updates install *over* the old app because every release is signed with the **same key**, so your login and data are kept.

Tech: `lib/core/update_checker.dart`, `package_info_plus`, `open_filex`.

Rule we follow: the version number must be bumped on every change — otherwise phones never see an update (documented in `CLAUDE.md`).

F. Reliability and security

F21 — Secure session storage

The login token is stored in the **Android Keystore** (hardware-backed encryption), not in plain files. A cached copy of your name and avatar (no secrets) lets the app open normally even without internet.

F22 — Network errors and offline behaviour

Every request goes through one wrapper with a **15-second timeout** (90 s for photo uploads) and turns low-level failures into readable messages ("No internet connection", "Server is taking too long"). Being offline never logs you out — the token is only removed if the server itself rejects it (HTTP 401).

F23 — Server-side protections (summary)

Risk	Protection
Guessing passwords	bcrypt hashing; rate limits; admin lockout after 8 failures
Stolen/forged tokens	Signed JWT; sessions checked against the database (suspended or reset users are cut off immediately)
Someone viewing others' cards	Private share tokens; DIGIPIN never unlocks data; same 404 for every unavailable link
Deleting someone else's photos	Image ids must lie inside your own Cloudinary folder
Website attacks	Cross-site request guard on admin calls (custom header + same-origin check), <code>SameSite=Strict</code> cookies
Misuse of the setup tools	Super-admin creation page only works on a developer's laptop

G. Website and admin

F24 — The public website

What: Not just a download page: it has a **DIGIPIN converter**, a public location page `/digipin/<PIN>` (map only — no private data), the private card page `/c/<token>`, an About page, and a full **dashboard** (log in and manage cards in a browser). Includes light/dark theme and an Android APK download prompt.

Tech: Next.js App Router, React, `framer-motion`, `lucide-react`.

F25 — Admin panel (website only)

What: A hidden control room for the people who run DigiRoutes.

How to open: press **Ctrl + Shift + K** on any page → sign-in box. Nothing on the site links to it; `/admin` shows a plain *404 Not Found* unless you are signed in.

Roles: | | Super admin (only one) | Sub-admin | | --- | --- | --- | | See analytics, users list, cards list | ✓ | ✓ | | Suspend / delete users, reset their password | ✓ | - | | Open a card's contents (recorded), remove cards | ✓ | - | | Create / disable / delete sub-admins, reset their passwords | ✓ | - | | Read the audit log | ✓ | - |

Pages: *Overview* (users, active users, cards, photos, link views, 30-day charts, categories, sharing split, Cloudinary storage) · *Users* · *Cards* · *Admins* · *Audit log* · *My account*.

Privacy rules: sub-admins never see photos, notes or phone numbers; DIGIPINs are masked; admins can't search by DIGIPIN. Every sign-in and every change is written to an **append-only audit log**.

Security design: admins are a *separate* database collection from users (signing up can never create an admin); separate cookie and signing key; 8-hour sessions checked against the database on every request (disabling an admin logs them out immediately); temporary passwords are random, shown once and must be replaced at first sign-in.

Tech: `digiroute/app/admin/*`, `app/api/admin/*`, `lib/adminAuth.ts`, models `Admin`, `AuditLog`.

Demo: Ctrl+Shift+K → sign in → Overview charts → Users → suspend a test user → show that the user's app session ends.

Teacher may ask: *Why is the shortcut hidden — is hiding enough?* → No, hiding is only convenience. The real protection is the server: strong passwords, rate limiting, lockout, signed cookies and role checks on every request.

F26 — Creating the super admin safely

What: The first admin can't be created through a normal sign-up.

How: A setup page (`/dev/setup-admin`) that works **only** when the site runs on a developer's own computer in development mode (`NODE_ENV=development`), not on Vercel/CI, and addressed to `localhost`. On the live server it answers 404. Only one super admin can ever exist. We tested that it is dead in a production build.

H. Engineering practice

F27 — Testing

- **39 automated tests** in the app (`flutter test`): DIGIPIN round trips and error cases, QR/link parsing, share-link behaviour, card model, route parsing, distance maths, error messages, pagination logic and a UI smoke test.
- During development we also ran scripted API checks (permissions for both admin roles, rate limits, share-link on/off/expiry/reset, image cleanup, account deletion) and real-browser runs of the admin panel.
- **Honest gap:** the website has no automated test suite checked into the repository yet (future work).

F28 — CI/CD, releases and versioning

What: Merging to `main` automatically publishes the product.

How:

```
merge to main
|
+--> GitHub Actions:  run tests --> build signed APK --> publish GitHub Release
|                      (file name: digiroutes-v1.6.0.apk)
|
+--> Vercel:          build website + API --> deploy to production
```

- **Semantic versioning** `MAJOR.MINOR.PATCH` : patch = bug fix, minor = new feature, major = breaking change.
- **Secrets** (signing keystore, routing key) are stored as GitHub secrets, never in the code.
- We use **one branch and one pull request per release**, deleted after merging.

F29 — Project structure (where things live)

```
digiroutes_app/lib/
  core/      theme, router, ApiHttp (network wrapper), routing service, update checker, widgets
  data/      models (AddressCard, AppUser), repositories (talk to the API), local storage
  logic/     providers (Riverpod), digipin.dart, qr_link.dart, geo.dart
  ui/        screens (one file per screen), widgets (QR sheet, avatar, dialogs), shell (bottom bar)

digiroute/
  app/       pages (website) and app/api (backend endpoints)
  components/ website + admin UI
  lib/       digipin, session, shareLinks, rateLimit, adminAuth ...
  models/    User, AddressCard, RateLimit, Admin, AuditLog
```

Part 5 — Backend API at a glance

Area	Endpoints
Accounts	POST /api/auth/signup, /login, /logout · GET/PUT/DELETE /api/auth/me
Cards (owner)	GET/POST /api/cards · GET/PUT/DELETE /api/cards/:id
Sharing (public)	GET /api/cards/shared/:token · GET /api/cards/digipin/:pin (owner / legacy only)
Photos	POST /api/upload/sign · POST /api/upload/cleanup
DIGIPIN helpers	GET /api/digipin/encode, /decode
Admin	/api/admin/auth/*, /stats, /users..., /cards..., /admins..., /audit

Part 6 — Database design (MongoDB)

User (1) -----< (many)	AddressCard	Admin (1) -----< (many)	AuditLog
name, email, passwordHash	owner -> User	role: super sub	who, what, when, IP
avatar, status,	digipin, title	passwordHash, status	
sessionVersion	photoUrls, photoIds		
	category, note, phone	RateLimit	
	shareToken, sharingEnabled	key, count	
	shareExpiresAt, hidePhone	(old rows deleted	
	viewCount, isFavorite	automatically)	

A TTL (time-to-live) index automatically deletes old `RateLimit` rows.

Part 7 — Problems we hit and how we solved them

(Great material for "challenges faced" in the viva.) | Problem | What happened | Solution | | --- | --- | --- | | **QR scanner crashed only in the release APK** | Android's code shrinker (R8) renamed classes Google ML Kit needs at start-up. It worked in debug, so we missed it. | Added keep-rules (`proguard-rules.pro`) and now test native features in release mode. | | **Privacy hole in sharing** | A DIGIPIN is computable by anyone, yet it unlocked a card. | Introduced secret share tokens, owner controls and a grace period for old links. | | **Photo ownership hole** | The server trusted any image id sent by the client. | Image ids must be inside the user's own folder. | | **Routing provider moved** | OpenRouteService deprecated its old host. | Read the notice, switched to the new host, and tested live. | | **Login lock-out of real admins** | The rate limiter counted successful sign-ins too. | Count only *failed* attempts. | | **Old tests broke CI** | A redesign changed text the smoke test looked for. | Update tests together with UI changes; always run `flutter test` before pushing. | | **Phone couldn't reach laptop backend** | The laptop's Wi-Fi address changed. | Rebuilt with the new address (and documented it). | | **Many stacked pull requests** | Several merges in a row each triggered their own build and release (extra versions nobody needed). | One consolidated pull request per repo per release. |

Part 8 — Limitations and future work

- Android only (iOS not built or tested). India only (DIGIPIN's coverage).
- Photo links are hard to guess but not *signed*; strict signed URLs would be stronger.
- No offline card cache yet; no two-factor login for the super admin; app has no "change password" screen yet (an admin reset gives a temporary password).
- Website lacks an automated test suite; no home-screen widget; APK is large (≈ 74 MB) — per-phone-type APKs would be $\sim 25\text{--}30$ MB.
- Package name is still `com.example...`; it must change before a Play Store release.

Part 9 — Glossary

Term	Meaning
DIGIPIN	10-character grid code for a ~4 m square of India (by India Post).
API	The set of web addresses the app calls to talk to the server.
JWT	A signed token proving who you are.
Hash (bcrypt)	A one-way scramble of a password.
REST / JSON	The style and data format used for API calls.
Cursor pagination	Loading a long list in pages using "the item after this one".
Rate limiting	Refusing too many requests from one source in a short time.
CI/CD	Automatic testing, building and deploying after every merge.
APK	The installable Android app file.
App Links	Android feature that opens verified web links directly in an app.
R8	Android's tool that shrinks and renames code in release builds.
CDN	A network of servers that delivers images fast.
Haversine	Formula for distance between two points on a sphere.

Part 10 — Suggested division of features

Adjust to your group size. Each part is ~3–4 minutes.

#	Presenter focus	Features	Key message
1	Problem, idea, DIGIPIN	Parts 1 & 3, F4, F5, F6	"Addresses are vague; DIGIPIN gives a 4 m code; here is the maths."
2	Accounts & app basics	F1, F2, F3, F17, F18, F19, F21, F22	"Secure sign-in, saved safely, works with poor internet."
3	Address cards	F8, F9, F10, F11	"A rich card for every doorstep, fast even with hundreds."
4	Sharing & scanning	F12, F13, F14, F15, F16	"Share only with people you choose; scan or tap to open."
5	Routes & maps	F7, F5 (map part)	"Distance, time and road route between any two points."
6	Backend, database, admin	Parts 2, 5, 6, F23, F24, F25, F26	"How the server protects data and how admins manage it."
7	Engineering	F20, F27, F28, F29, Parts 7 & 8	"Updates, testing, automatic releases, lessons learned."

Part 11 — A 12-minute live demo script

1. **(1 min)** Open the app: splash → onboarding slides → sign up.
2. **(1 min)** Home → *Get My DIGIPIN*; Compass → decode **39J438TJC7**.
3. **(2 min)** + → create a card (category, note, photo) → it appears in *My Cards*.
4. **(1 min)** Search, favourite and filter cards.
5. **(2 min)** Open the card → *Share with a link* → show the QR → scan it with a friend's phone / pick it from the gallery.
6. **(1 min)** Turn sharing off → the same link now says "not available" → *Reset link*.
7. **(1 min)** *Where to?* → choose a place → distance, time, route.
8. **(1 min)** Profile and Settings (theme, delete-account warning, update check).
9. **(2 min)** On the website: Ctrl+Shift+K → admin overview, a user list, the audit log. Then show GitHub **Releases** (the auto-built APK).

Tip: keep a second phone logged out for the "visitor" part, and take screenshots of each step as a backup in case the internet is slow.

Part 12 — Questions the teacher may ask (general)

1. **Why did you choose Flutter?** One codebase, smooth animations, plugins for camera/GPS/QR.
 2. **Why a separate backend?** Accounts, shared cards and admin tools need a server and a database; the phone alone can't share data between people.
 3. **How is a password protected?** Hashed with bcrypt; only the hash is stored; login attempts are rate limited.
 4. **What happens if the internet drops?** Requests time out with a clear message; you stay logged in; DIGIPIN maths still works offline.
 5. **Can a stranger find my address?** Not through DIGIPIN. Only someone with your secret link/QR can open a card, and you can switch it off, reset it or expire it.
 6. **What if the QR is photographed by someone else?** They can open the card until you reset or expire the link — that is why those controls exist.
 7. **How do you stop abuse?** Rate limiting, suspension of users by the admin, audit log, removal of content.
 8. **How do updates reach users?** The app checks GitHub Releases and installs a newer signed APK.
 9. **What does it cost?** Nothing — Vercel, MongoDB Atlas, Cloudinary, OpenRouteService and GitHub are used on free plans.
 10. **How did you test it?** 39 automated app tests, scripted API checks, and manual runs on a real Android phone in release mode.
 11. **What would you improve next?** Two-factor login, offline cache, signed photo URLs, iOS version, Play Store release.
 12. **What is the difference between the sub-admin and super admin?** Sub-admins can only look at statistics and lists; the super admin can change data and manage other admins.
-

Appendix — Running the project locally

App: `cd digiroutes_app && flutter pub get && flutter run` (use `--dart-define-from-file=env.local.json` for the routing key, and `--dart-define=API_BASE_URL=http://<laptop-ip>:3000` to use a local backend).

Website/API: `cd digiroute && npm install && docker compose up -d && npm run dev` (needs `.env.local` with `MONGODB_URI`, `SESSION_SECRET`, Cloudinary keys — see `sample.env`).

Checks: `flutter analyze && flutter test (app) · npx tsc --noEmit && npm run build (web)`.

DigiRoutes v1.6.0 — minor project documentation.